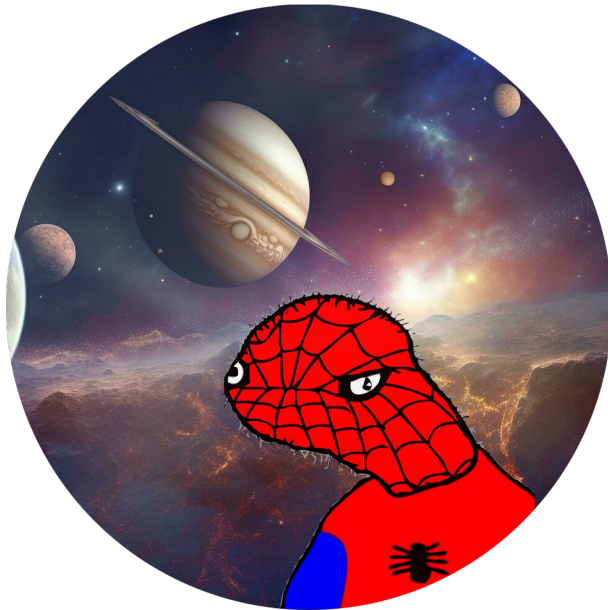




Privacy Cash - Privacy Cash Core EVM Audit Report

Version 1.2

Zigtur

March 17, 2026

Privacy Cash - Privacy Cash Core EVM - Audit Report

Zigtur

March 17, 2026

Prepared by: Zigtur

Table of Contents

- Table of Contents
- Introduction
 - Disclaimer
 - About Zigtur
 - About Privacy Cash
- Security Assessment Summary
 - Scope
 - Deployment chains
 - Risk Classification
- Issues
 - LOW-01 - Fees are bypassable
 - LOW-02 - Fully private transfers are not allowed due to `_extAmount` zero check
 - INFO-01 - `MerkleTreeWithHistory` storage is not 50-slot aligned
 - INFO-02 - `MerkleTreeWithHistory` is still using the zero hash value from Tornado Nova
 - INFO-03 - UUPS proxy is used and not `TransparentUpgradeableProxy`
 - INFO-04 - Cross-chain proof reuse in case of low entropy random

Introduction

Disclaimer

A smart contract security review cannot guarantee the complete absence of vulnerabilities. This effort, bound by time, resources, and expertise, aims to identify as many security issues as possible. However, there is no assurance of 100% security post-review, nor is there a guarantee that the review will uncover all potential problems in the smart contracts. It is highly recommended to conduct subsequent security reviews, implement bug bounty programs, and perform on-chain monitoring.

About Zigtur

Zigtur is an independent blockchain security researcher dedicated to enhancing the security of the blockchain ecosystem. With a history of identifying numerous security vulnerabilities across various protocols in public audit contests and private audits, **Zigtur** strives to contribute to the safety and reliability of blockchain projects through meticulous security research and reviews. Explore previous work [here](#) or reach out on X [@zigtur](#).

About Privacy Cash

PrivacyCash is a decentralized protocol for secure and anonymous transactions built on Solana. It is now expanding to EVM chains.

Using zero-knowledge proofs, it lets users deposit assets and withdraw them to different addresses, breaking the link between sender and receiver. By combining advanced cryptography with decentralized infrastructure, PrivacyCash restores financial privacy and freedom on public blockchains.

Security Assessment Summary

Review commit hash - 26f8602

Fixes review commit hash - f5a6758

Scope

- contracts/* (excluding Mocks)
- scripts/deployTestnet.js

- circuits/*
- build/circuits/*

Deployment chains

- Base

Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	High	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Issues

LOW-01 - Fees are bypassable

Scope:

- EtherPool.sol#L110-L113

Description

The protocol does not enforce any mandatory fee at the contract level. The `fee` and `feeRecipient` fields are entirely user-controlled as part of the `ExtData` struct:

```
struct ExtData {
    address recipient;
    int256 extAmount;
    address feeRecipient;
    uint256 fee;
    bytes encryptedOutput1;
    bytes encryptedOutput2;
}
```

While these values are committed to the ZK proof through `extDataHash` (preventing tampering after proof generation), the user who generates the proof can always set `fee = 0` or set `feeRecipient` to their own address and submit the transaction themselves.

The fee transfer logic only executes when `fee > 0`:

```
if (_extData.fee > 0) {
    (bool success, ) = _extData.feeRecipient.call{value: _extData.fee}("");
    require(success, "Fee transfer failed");
}
```

This means there is no mechanism for the protocol to enforce a mandatory fee on transactions. In the relayer model, while the relayer can refuse to submit a transaction that does not include a sufficient fee, users can always bypass the relayer entirely and submit their own transactions with zero fee. This reduces the economic incentive for relayers and prevents the protocol from collecting fees for sustainability.

Note that bypassing the relayer by submitting the transaction directly reduces privacy, since the submitter's address is publicly visible on-chain.

Recommendation

If the protocol intends to collect mandatory fees, consider adding a protocol-level minimum fee enforced in the `transact()` function, with a protocol-owned `feeRecipient` address. For example:

```
uint256 public protocolFee;           // set by admin
address public protocolFeeRecipient; // set by admin

// In transact():
require(_extData.fee >= protocolFee, "Fee below minimum");
```

If fees are intentionally optional (relayer-only tips), this behavior should be documented as expected.

Privacy Cash

Fixed. A comment has been added in PR1 to indicate that fees are not mandatory.

Zigtur

Fixed.

LOW-02 - Fully private transfers are not allowed due to `_extAmount` zero check

Scope:

- EtherPool.sol#L141
- EtherPool.sol#L106-L108

Description

The `calculatePublicAmount()` function requires that `_extAmount` is strictly non-zero:

```
function calculatePublicAmount(int256 _extAmount, uint256 _fee) public pure returns
↳ (uint256) {
    require(_fee < MAX_FEE, "Invalid fee");
    require(_extAmount > -MAX_EXT_AMOUNT && _extAmount < MAX_EXT_AMOUNT, "Invalid
↳ ext amount");
    require(
        (_extAmount > 0 && uint256(_extAmount) > _fee) ||
        (_extAmount < 0 && uint256(-_extAmount) > _fee),
        "ext amount must exceed fee for deposits"
    );
    uint256 publicAmount = _extAmount - int256(_fee);
    return (publicAmount >= 0) ? uint256(publicAmount) : FIELD_SIZE -
↳ uint256(-publicAmount);
}
```

This check enforces that `_extAmount` cannot be zero (since neither `_extAmount > 0` nor `_extAmount < 0` would be satisfied). This prevents fully private (shielded) transfers, where a user reshuffles UTXOs within the pool without any ETH entering or leaving.

In a typical UTXO-based shielded pool, three transaction types are supported:

1. **Deposit** (`extAmount > 0`): ETH enters the pool.
2. **Withdrawal** (`extAmount < 0`): ETH leaves the pool.
3. **Shielded transfer** (`extAmount = 0, fee = 0`): UTXOs are consumed and new UTXOs are created, but no ETH moves in or out of the contract. This is used to transfer value between users privately.

The current constraint makes type 3 impossible, rendering the following branch in `transact()` unreachable dead code:

```
} else {
    require(msg.value == 0, "Cannot send ETH for zero-amount tx");
}
```

Recommendation

If shielded transfers are an intended feature, update the check in `calculatePublicAmount()` to allow `_extAmount == 0` with `_fee == 0`:

```
require(
  (_extAmount == 0 && _fee == 0) ||
  (_extAmount > 0 && uint256(_extAmount) > _fee) ||
  (_extAmount < 0 && uint256(-_extAmount) > _fee),
  "ext amount must exceed fee"
);
```

If shielded transfers are intentionally disallowed, remove the dead code in the `else` branch and document this design decision.

Privacy Cash

Fixed. A comment has been added in PR1 to indicate that internal transfers are not allowed.

Zigtur

Fixed. The `msg.value == 0` edge-case has been removed.

INFO-01 - MerkleTreeWithHistory storage is not 50-slot aligned

Scope:

- MerkleTreeWithHistory.sol#L22-L26
- MerkleTreeWithHistory.sol#L150

Description

The `MerkleTreeWithHistory` contract is used as a base contract in an upgradeable proxy architecture. When using inheritance with upgradeable contracts, a common convention is to reserve storage gap slots so that each base contract occupies a fixed, aligned number of storage slots (typically a multiple of 50). This ensures that adding new storage variables to a base contract in future upgrades does not collide with storage slots used by derived contracts.

`MerkleTreeWithHistory` uses 3 storage slots for its state variables (excluding `immutable` and `constant` variables which are not stored in contract storage):

Slot	Variable(s)
0	<code>filledSubtrees</code> (mapping)
1	<code>roots</code> (mapping)
2	<code>currentRootIndex</code> (uint32) + <code>nextIndex</code> (uint32), packed

The contract then declares a `__gap` of 50 slots:


```
uint256[50] private __gap;
```

This results in a total of **53 storage slots** (3 + 50), which is not aligned to the conventional 50-slot boundary. The gap should be 47 slots (`uint256[47] private __gap`) to bring the total to 50.

Note that the `Initializable` base contract from OpenZeppelin uses ERC-7201 namespaced storage since v5 and does not consume regular storage slots.

Recommendation

Change the storage gap to ensure `MerkleTreeWithHistory` occupies exactly 50 storage slots:

```
uint256[47] private __gap;
```

Privacy Cash

Fixed. PR2 fixes it by using 47 as the gap length.

Zigtur

Fixed.

INFO-02 - `MerkleTreeWithHistory` is still using the zero hash value from Tornado Nova

Scope:

- `MerkleTreeWithHistory.sol#L114`

Description

The `zeros()` function in `MerkleTreeWithHistory` contains 33 hardcoded Poseidon hash values representing empty subtree hashes at each level of the Merkle tree. The seed value at index 0 is:

```
0x2fe54c60d3acabf3343a35b6eba15db4821b340f76e741e2249685ed4899af6c
```

This corresponds to the decimal value

```
21663839004416932945382355908790599225266501822907911457504978515578255421292
```

, which is `keccak256("tornado") % FIELD_SIZE`. This is the exact zero leaf value used in Tornado Cash Nova, from which this contract was forked.

While the zero values are computed correctly (each level is `poseidon(zeros[i-1], zeros[i-1])`), using a domain string from another protocol ("tornado") is not ideal. In practice, the security impact is negligible since the zero element simply represents an empty leaf and the Merkle tree is initialized deterministically from it. There is no cross-protocol attack vector because the contracts are deployed independently with separate state. However, using a protocol-specific domain string (e.g., `keccak256("privacy-cash") % FIELD_SIZE`) would be a cleaner approach and eliminate any confusion about the codebase provenance.

Recommendation

Consider replacing the zero leaf element with a Privacy Cash-specific domain string, such as `keccak256("privacy-cash") % FIELD_SIZE`, and recomputing all 33 zero values accordingly.

Note that this change requires updating both the Solidity contract and the off-chain code (test files, SDK, circom circuits) that reference `MERKLE_TREE_ZERO_VALUE`.

Privacy Cash

Fixed. PR3 implements the use of a privacy cash dedicated value.

Zigtur

Fixed. All poseidon hashes for zero value are correct.

INFO-03 - UUPS proxy is used and not TransparentUpgradeableProxy

Scope:

- EtherPool.sol#L10
- deployTestnet.js#L28-L37

Description

The EtherPool contract uses the **UUPS (Universal Upgradeable Proxy Standard)** proxy pattern by inheriting UUPSUpgradeable from OpenZeppelin:

```
contract EtherPool is MerkleTreeWithHistory, UUPSUpgradeable {
```

The deployment script deploys it behind an ERC1967Proxy with kind: 'uups'.

There are two main upgradeable proxy patterns available in OpenZeppelin:

	UUPS	TransparentUpgradeableProxy
Upgrade logic location	Implementation contract	Proxy contract
Upgrade authorization	<code>_authorizeUpgrade()</code> in implementation	Dedicated ProxyAdmin contract
Deploy cost	Lower (simpler proxy)	Higher (proxy includes upgrade logic)
Per-call gas cost	Lower	Higher (admin vs. user check on every call)
Risk	If a new implementation omits <code>_authorizeUpgrade()</code> or UUPSUpgradeable, the contract becomes permanently non-upgradeable	No such risk; upgrade logic is always in the proxy

The main operational risk with UUPS is that upgradeability can be accidentally lost. If a new implementation contract is deployed without properly inheriting UUPSUpgradeable or without including the `_authorizeUpgrade()` function, the upgrade mechanism is permanently bricked.

In the current codebase, `_authorizeUpgrade()` is correctly implemented with the `onlyAdmin` modifier:

```
function _authorizeUpgrade(address) internal override onlyAdmin {}
```

Recommendation

Ensure that every future implementation contract deployed as an upgrade:

1. Inherits `UUPSUpgradeable`.
2. Overrides `_authorizeUpgrade()` with proper access control.
3. Is thoroughly tested with OpenZeppelin's upgrade safety validation tooling (`@openzeppelin/hardhat-upgrades`).

Consider adding integration tests that verify upgradeability is preserved after each upgrade (as demonstrated by the existing `EtherPoolV2Mock` test). Alternatively, if the upgrade risk is deemed unacceptable, consider switching to `TransparentUpgradeableProxy` which keeps the upgrade logic in the proxy itself.

Privacy Cash

Acknowledged.

Zigtur

Acknowledged.

INFO-04 - Cross-chain proof reuse in case of low entropy random

Scope:

- transaction.circom#L65-L67
- transaction.circom#L101-L106

Description

A UTXO commitment is computed as `poseidon(amount, pubkey, blinding, mintAddress)`, where `blinding` is a random value generated off-chain and `mintAddress` is hardcoded to `0` for native ETH on all EVM chains. The commitment structure does not include any chain-specific identifier (e.g., chain ID, contract address).

```
// commitment = hash(amount, pubKey, blinding, mintAddress)
inCommitmentHasher[tx] = Poseidon(4);
inCommitmentHasher[tx].inputs[0] <== inAmount[tx];
inCommitmentHasher[tx].inputs[1] <== inKeypair[tx].publicKey;
inCommitmentHasher[tx].inputs[2] <== inBlinding[tx];
inCommitmentHasher[tx].inputs[3] <== mintAddress;
```

The `blinding` factor is currently generated using `crypto.randomBytes(31)` (31 bytes of cryptographic randomness), which provides adequate entropy under normal conditions. However, if Privacy Cash is deployed on multiple EVM chains and the randomness source produces a collision (whether due to a weak environment, a faulty PRNG, or an implementation error in a third-party SDK), two issues arise:

1. **Cross-chain UTXO correlation:** If the same `blinding` value is used for deposits on two different chains with the same `amount` and `keypair`, identical commitments appear on both chains. An on-chain observer can trivially correlate these deposits and link them to the same user, defeating the privacy guarantees of the protocol.
2. **Deposit proof replayability:** For deposit transactions, input UTXOs are zero-amount and the Merkle proof verification is disabled (the circuit sets `inCheckRoot.enabled <== inAmount[tx]`, so zero-amount inputs skip the root check). Immediately after deployment, both chains share the same initial Merkle root (the empty tree root derived from the same zero values). A valid deposit proof generated for one chain could therefore be submitted on another chain with the same `msg.value`, resulting in duplicate commitments being inserted into both trees. While this does not steal funds (the depositor pays ETH on each chain), it creates identical UTXO states across chains, which could lead to unintended consequences.

Mitigating factors:

- The current SDK implementation uses `crypto.randomBytes(31)`, which provides 248 bits of entropy — collisions are astronomically unlikely in practice.

- After the first transactions on each chain, the Merkle roots diverge and proof replay is no longer possible.
- The `extDataHash` binds the proof to specific external data, but this data also lacks chain-specific fields.

Note: This is unlikely to occur and is reported for exhaustivity.

Recommendation

To provide defense-in-depth against potential randomness issues and formally prevent cross-chain proof correlation:

1. **Include chain-specific data in the commitment:** Add the chain ID or contract address as an input to the commitment hash in the circuit, so that identical parameters on different chains produce different commitments. This is the most robust solution but requires a circuit change.
2. **Document randomness requirements:** The SDK and any third-party integration documentation should clearly state that the `blinding` value must be generated from a cryptographically secure PRNG with at least 248 bits of entropy, and must never be reused across UTXOs or chains.

Privacy Cash

Acknowledged.

Zigtur

Acknowledged.